Formal Grammars and Backus-Naur Forms

Human languages are very difficult to describe. Suppose you are reading a story, and you come across the following full sentence:

"Great."

What does the speaker mean?

Formal Grammars and Backus-Naur Forms

Fortunately, programming languages do not have nuance. So we can use formal grammars to define the syntax of programming languages.

The Backus-Naur Form (BNF) is a popular tool, which is used to specify programming languages for compilation.

Tokens

Formal languages are made of tokens (sometimes called "terminal symbols"). These are the irreducible elements of a program. In C, this means:

- Numbers and character constants
- Operators and punctuation
- Strings of text
- Reserved words

In an expression like sum = x+next;, the tokens are sum, =, x, +, next, ;. Even though sum and next are multiple characters, they are treated as indivisible units.

Syntax

Backus-Naur form builds complex elements from simple building blocks. Basically, the whole system is this:
<> Angle brackets define elements of the grammar.
<- Some kind of assignment operator to define an element. Sometimes := or :- text
terminals Anything outside angle brackets is part of the language
| means OR

Our first BNF

Consider the following:
<string> ← <astring>|<astring><bstring>
<astring> ← a|a<astring>
<bstring> ←b|b<bstring>

What does this define?

Basically, this is the set of all strings that consists of one a followed by zero or more b:
Valid: a, ab, aaaaaaa, aaaabbbbbb…
Invalid: b, ba, baaaaa, aba, aaaabbbbaaa…

A more interesting BNF

Consider the following:

<id> ← <letter><restofid>|_<restofid>

<restofid> ← <validchar>|<restofid><validchar>

<validchar> ←<letter>|<digit>|_

<letter> ←a|b|c|…|y|z|A|B|…|Y|Z

<digit> ←1|2|3|4|5|6|7|8|9|0

What does this one define?

Basically, this defines all legal identifiers in C/C++/Java.

An identifier must start with a letter (uppercase or lower case) or an underscore.

After the first character, any letter, digit or underscore is acceptable.

Even more interesting BNF

Consider the following:

<assign> ← <id>=<expr>

<id> ← A|B|C

<expr> ← <id>+<expr>
|<id>*<expr>
|(<expr>)
|<id>

What does this one define?

This language is simple, but it provides insight.
Consider the features of the language:

Recursion The definition of <expr> includes <expr>. This is a common way to say “expressions can be made from other expressions”

Syntax This is the syntax of a simple math-like language that operates on variables called A,B,C

Parse tree The BNF can turn the string of tokens into a parse tree that captures the meaning of the expression

This one defines mathematical expressions. We can write things like:

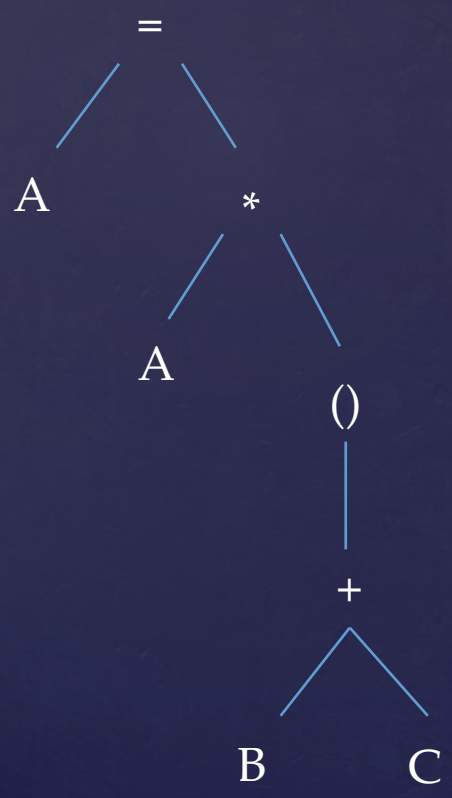
A=B+C

B=A*C

B=A*A

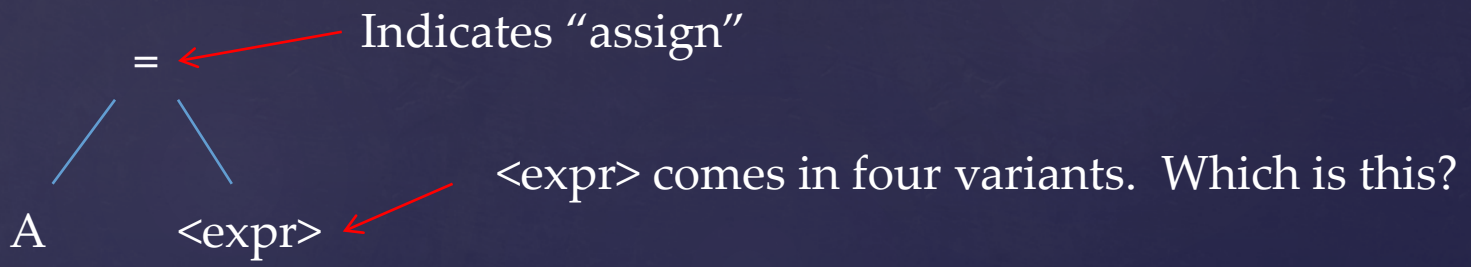
A=B+(C*B)

If we start with A=A*(B+C) we can make the following parse tree:



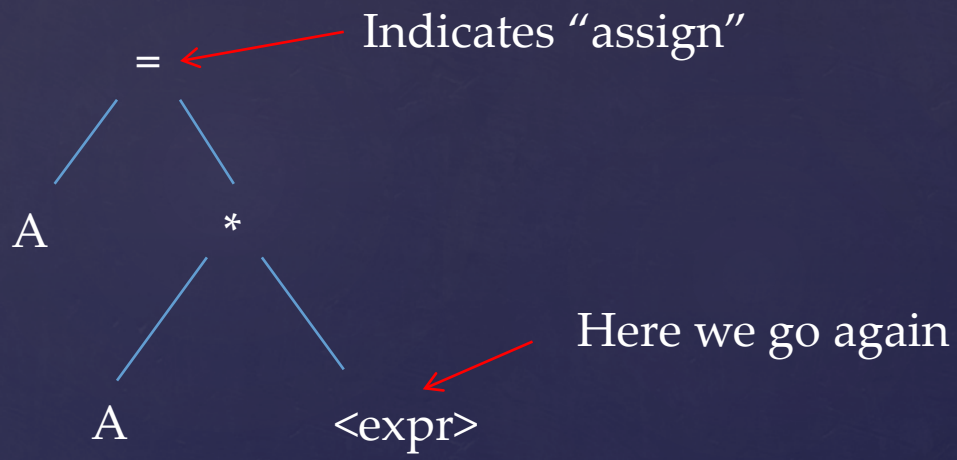
The parse tree grows as we read the expression

If we start with A=A*(B+C) we can make the following parse tree:



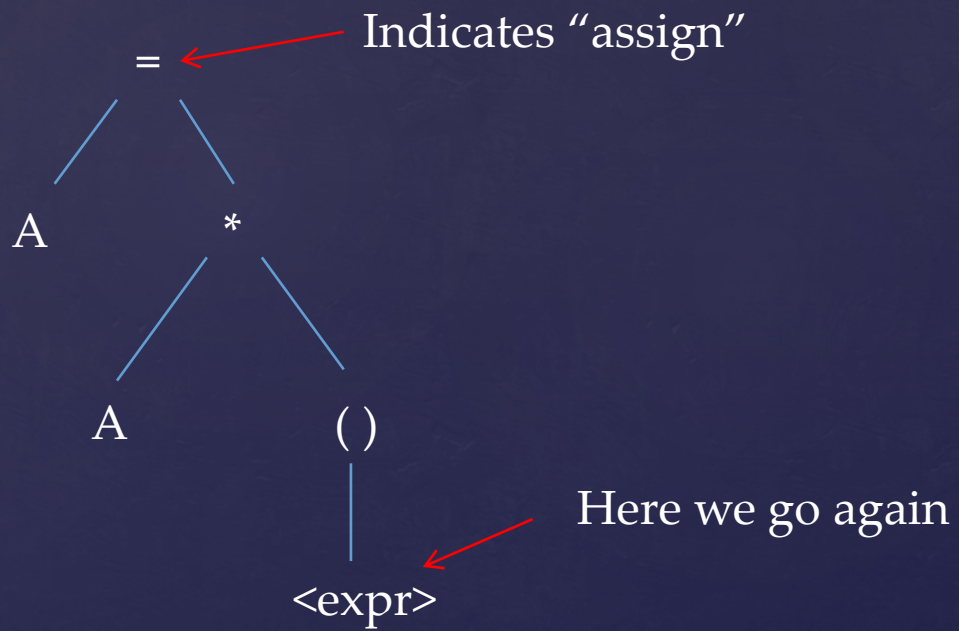
According to the language, = assigns <expr> to <id>. A is a valid <id>, but now we need to parse the expression

If we start with A=A*(B+C) we can make the following parse tree:



The next token is an <id>, then we find * indicating which kind of expression it is.

If we start with A=A*(B+C) we can make the following parse tree:



The parenthesis indicates that whatever is in the brackets parses to an <expr>. So we keep going.

Let’s do another one:

Form the parse tree for A=A*B+C*(A+B).

Slightly improved

Consider the following:

<assign> ← <id>=<expr>

<id> ← A|B|C

<expr> ← <expr>+<term>
|<term>

<term> ← <term>*<factor>
|<factor>

<factor> ← <id>
|(<expr>)

Now try A=A*B+C*(A+B)

Compilation

Now that we have a basic understanding of BNF, we can start to think about what compilers do.

Again I stress that we will only scratch the surface. Much of what modern compilers do is optimization to take advantage of CPU architectural features. We will instead focus on the basics.

I will mostly focus on compiling C for the S4 (or something like it).

Compilation – Key concepts

There are two key concepts we will need in our discussion:

Simplicity: Basically this is an attempt to describe how "difficult" the language is to learn. It can mean size (how many elements does the language contain) or multiplicity (how many ways are there to do a given thing)

Is C large or small?

Most people consider C to be a small language, since it has a small number of reserved words.

However, in order to do anything useful with C requires a large set of built-in library functions, so…

What about multiplicity? Well, consider this: How many ways can you increment x?

x=x+1; x++; x+=1; ++x; Others?
Are these all the same?

Obfuscated C

C is in fact so complicated and allows so much freedom that a subculture of "Obfuscated C" grew up, with programmers challenging one another to write the most complicated (but functioning) garbage they could imagine. Even non-obfuscated C can contain meaningless and unnecessary complexity:

```
void strcpy(char * d, char * s)
{
    while (*d++=*s++);
}
```

```
if (p=(S*)malloc(sizeof(S)))
{
    // Use the pointer
}
```

Some show off once proved that every C program could be coded as one for loop. Good luck debugging it.

Compilation – Key concepts

Orthogonality: Officially, a language is orthogonal if it has a small set of structures combined in a small set of ways.

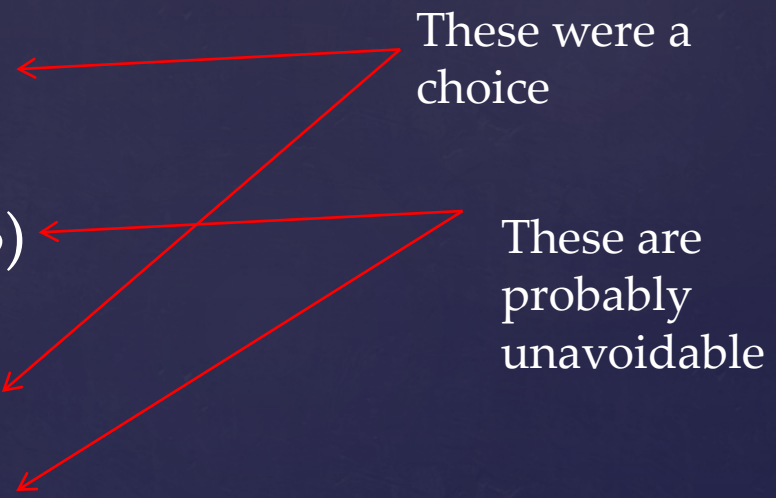
Unofficially it means "how many special cases are there"?

The extreme example of an orthogonal language is functional languages like LISP, that make no distinction between code, data, functions, expressions, conditions…

Orthgonality in C

C is considered fairly orthogonal, but there are places where it breaks down. Consider:

&x	y&x
(x+3)	f(x+3)
x*=y	x=*y
-3	x-3



Orthgonality in C

Do we even want orthogonality? All expressions are equal. This causes no end of problems.

```
void strcpy(char * d, char * s)
{
    while (*d++=*s++);
}
```

```
if (p=(S*)malloc(sizeof(S)))
{
    // Use the pointer
}
```

```
if (x=5)
{
    // Do something
}
```

R-values and L-values

To begin thinking about compilation, we need to understand r-value and l-value.

Consider the expression: x=x+1

Crazy, right? x does NOT equal x+1. That's like the first thing we learn in math class.

It works for us though, because we understand that the x on the left is just different from the x on the right.

R-values and L-values

This is the reason why all programming languages have different operators for assignment (=) and comparison (==).

They are not the same.

They don't even operate on the same kinds of things.

For our purposes, an l-value is the thing on the left side of the assignment expression. You can think of this as the address of the expression. Variables declared const have l-values, even though you can't write to them.

An r-value is the thing on the right side of the assignment expression. You can think of this as the value of the expression.

Some operators provide r-values ONLY. Arithmetic operators and the "address of" (&) operator do not have l-values.

Most expressions in C have r-values, although it is debatable whether structs do. You can write s1=s2 (assign one struct to another struct) or call f(s1) (push s1 onto the stack). These are legal, but really it's shorthand for an entire algorithm, not just a value.

C vs C++

One interesting difference between C and C++ is the value type of assignments. In C, the assignment operator returns an r-value. In C++, the assignment operator returns an l-value.

Both languages let you do this: x=y=z

Only C++ lets you do &(x=y), (x=y).member=z and so on.

Obligatory reminder: just because you CAN do a thing, it does not follow that you SHOULD do that thing.

Now that we have r-values and l-values, we can start compiling!

Consider the simple expression: x=y+z;

This means: Compute the r-value of (y+z) and put it at the l-value of x

Remember, parse trees are recursive. So when we ask "what is the r-value of (y+z)", it really means:

1. Compute the r-value of y and store it somewhere
2. Compute the r-value of z and store it somewhere
3. Add them

In a stack-based architecture like the S4, "somewhere" is the top of the stack. So we could do this to compile x=y+z:

```
A←r-value(y)        ; Compute r-value of y
PUSHA               ; Put it on the stack
A ←r-value(z)       ; Compute r-value of z
ADDA                ; Pop into MDR, A ←A + MDR
PUSHA               ; A is now y+z.  Save it for later
X ←l-value(x)       ; Compute l-value of x
POPA                ; Retrieve A
STAX 0              ; (X) ←A.  This puts y+z at x
```

Here's where orthogonality is your friend.

What does the assembly code for: *x=f(5)+g(4); look like?

It looks exactly the same. It's just that the part about "compute the r-value" and "compute the l-value" gets way more complicated.

Arrays in C

Arrays in C are a bare-bones implementation of a collection of objects. If you put int x[10]; into your program, the compiler understands:
"Set aside a continuous block of memory sufficient to store 10 ints. x is the address of the first element in the block"

Arrays in C

When you write x[4] the compiler computes *(x+4).
That means:

1. Multiply 4 × sizeof(int)
2. Add the result to x. That's your l-value.
3. If you need the r-value, dereference the l-value

For something like this (a constant index), the compiler will do the math, so the code generated should be identical to that for an int.

If the index is variable (x[i]) it has to do the math at runtime.

Structs in C

Structures are implemented basically the same as arrays. The differences are:

1. Structs can store elements of different sizes
2. Struct offsets are always constant

Consider the following:

```
struct S{
        int x;
        char c;
        double d;
        int array[10];
} s;
```

Structs in C

Suppose ints require four bytes, chars require one byte and doubles require eight bytes. Further, suppose that ints and doubles need to be stored on boundaries that are evenly divisible by four and eight, respectively.

Then probably we need:
4+1+3(wasted)+8+10*4=56 bytes to store this struct.

Structs in C

Now if you type s.d, here's what the compiler does

1. Compute the address s+8. That's your l-value.
2. Dereference the l-value if you need the r-value

Since the compiler knows the offsets ahead of time, there should be no difference in access time to this double than to any other double in the same scope.

Structs in Embedded Systems

As an aside, a smart cross-compiler for embedded development should let you define a struct to match any built-in memory mapped peripherals.

Suppose you have a serial port that has three one-byte registers, memory mapped as follows:

Register Name	Address
RxData	$100
TxData	$101
Status	$102

Structs in Embedded Systems

You should be able to define a struct like this:

```
struct SerialPort
{
    char RxData;
    char TxData;
    char Status;
};
```

And a pointer like this:

```
struct SerialPort * const p = (SerialPort *)0x100;
```

You can then access the registers through pointer p, as in:

```
p->TxData = 'a';
```

It looks complicated, but a smart compiler should generate code equal to a direct access.

Generating code for branches

Recall the BNF for **if**:

if(<expr>) <stat1> else <stat2>

The red bits are part of the language. The <white> bits are determined by your program.

Of course else is optional, and <stat> can be a block statement.

Here we see orthogonality. The conditional is just an expression, like any other. That's how all those crazy side effects are possible.

Generating code for branches

So what does the compiler do? It's just a matter of r-values and l-values. We would get something like this:

```
        A←r-value(<expr>)   ; Compute the expression
        TESTA               ; Set condition codes
        BEQ Else            ; Jump if FALSE
        <stat1>             ; Do the "if" part
        BRA End
Else:   <stat2>
End:                        ; Continue the program.
```

Generating code for branches

This compiler will work, but you would NOT like it. Suppose you do something simple, like: if (x>5).

You are probably expecting a direct comparison of x to 5, not a bunch of branches to compute whether or not x<5 is 1 or 0.

Fortunately, modern compilers are smart enough to do what you expect.

Generating code for branches

What about a top-tested loop (while)? The BNF is:

while(<expr>) <stat>

Generating code for branches

An implementation would look like this:

```
Top:    A←r-value(<expr>)   ; Compute the expression
        TESTA               ; Set condition codes
        BEQ End             ; Jump if FALSE
        <stat>              ; Do the loop
        BRA Top             ; Do it all again
End:                        ; Continue the program.
```

Generating code for branches

What about a bottom-tested loop (do..while)? The BNF is:

do <stat> while(<expr>)

Generating code for branches

Finally, we have the for loop. The BNF is:

for (<expr1>;<expr2>;<expr3>) <stat>

Any of <expr1>, <expr2>, <expr3> can be blank.

Generating code for branches

for is compiled as if you had written

```
<expr1>
while(<expr2>)
{
        <stat>
C:      <expr3>
}
```

Generating code for branches

for is complicated because of two special reserved words continue and break. For all the loops, break just means BRA End

But continue is different. For while and do, continue is BRA Top. In a for loop, continue is BRA C. <expr3> must happen.

switch

C also supports the switch statement. The BNF is basically this:

switch(<expr>)
{
case <constexpr1>: <stmt1>
case <constexpr2>: <stmt2>
…
default: <stmtD>
}

switch

switch is very primitive. If is almost, but not quite:
if (<expr>==<constexpr1>)<stmt1>
else if (<expr>==<constexpr2>)<stmt2>
…
else <stmtD>

But it is not that, because of what we call “fall through”. If you don’t put break at the end of <stmt1>, <stmt2> also happens.

You could do it with if…else… style only if you were willing to implement explicit gotos for each clause.

switch

As a result, switch is normally implemented as a jump table. The r-value of <expr> is used to index into a table of jump instructions that jump to a specific piece of code.

The fall-through then happens naturally.

If the constant expressions are far apart, jump tables don't make sense, because you need an entry for each value.

Function calls

Finally, we have function calls. We have basically already seen how these are implemented:

<callable>(<expr1>,<expr2>,…<exprn>)

Normally <callable> is a function name, but it can be any expression that evaluates to a function type.

Function calls

The compiler would do something like this:

A←r-value(<exprn>)
PUSHA
…
A ←r-value(<expr2>)
PUSHA
A ←r-value(<expr1>)
PUSHA
X ←l-value(<callable>)
JSR X